

Centro Universitário Uni Academia
Curso de Engenharia de Software
Disciplina: Inteligência Artificial e Machine
Learning

Data: 15/06/2026

Autores: Italo Butinholi Mendes, Gustavo
Correa, Vitorio Ribeiro, Lucas Nascif e Mateus
Cacilhas

Desenvolvimento de um Agente
Conversacional Híbrido (CopaBot)
utilizando Processamento de Linguagem
Natural e Arquitetura RAG

1. Introdução

A evolução da Inteligência Artificial (IA) tem transformado a forma como interagimos com sistemas computacionais, com destaque para os agentes conversacionais (chatbots). No entanto, o uso exclusivo de Grandes Modelos de Linguagem (LLMs) apresenta desafios práticos, como o alto custo computacional, latência nas respostas e a propensão a "alucinações" (geração de fatos incorretos).

Para mitigar esses problemas, o presente trabalho propõe o desenvolvimento do "CopaBot", um agente conversacional especializado em dados da Copa do Mundo. A arquitetura proposta é híbrida: utiliza processamento determinístico via *Natural Language Toolkit* (NLTK) para consultas rápidas e exatas em uma base de dados estruturada local, acionando uma arquitetura de Geração Aumentada por Recuperação (RAG) integrada à API da Hugging Face apenas como mecanismo de contingência (*fallback*) para perguntas complexas.

2. Fundamentação Teórica

A construção do agente baseia-se em conceitos fundamentais de Processamento de Linguagem Natural (PLN) e Aprendizado Profundo (*Deep Learning*).

- **PLN, Tokenização e Análise Semântica:** O PLN permite que o sistema compreenda o texto inserido pelo usuário. Para superar as limitações de expressões regulares (*Regex*), implementamos um *pipeline* com NLTK que realiza a tokenização (divisão da frase em unidades menores) e a remoção de *stopwords* (termos sem carga semântica isolada). Isso permite extrair entidades essenciais, como datas (ex: "2002") e localidades (ex: "Brasil").
- **Arquitetura RAG (Retrieval-Augmented Generation):** É uma técnica que aprimora LLMs ao conectar o modelo generativo a uma base de dados externa e confiável, garantindo que as respostas sejam ancoradas em fatos reais e atualizados, reduzindo significativamente as alucinações.
- **Transformers e LLMs:** Modelos modernos de linguagem baseiam-se na arquitetura *Transformer*, que utiliza atenção (*self-attention*) para processar o contexto de palavras simultaneamente, permitindo a geração de textos fluidos e altamente contextuais.

3. Estado da Arte e Desafios Técnicos

Atualmente, o mercado transita de chatbots baseados puramente em árvores de decisão para agentes baseados em Inteligência Artificial Generativa. Contudo, está transição esbarra em três desafios técnicos principais:

1. **Custos Computacionais e Latência:** Consultar um LLM na nuvem para cada interação gera um tempo de resposta (latência) frequentemente superior a 3 segundos, prejudicando a experiência do usuário.
2. **Alucinação:** LLMs genéricos tendem a inventar dados quando não possuem a informação exata.

O nosso projeto aborda o Estado da Arte ao resolver esses problemas simultaneamente: a consulta primária via NLTK na base local possui latência de milissegundos e custo zero. O LLM só é ativado quando estritamente necessário, e, mesmo assim, ancorado pela técnica RAG para evitar alucinações. Para lidar com a latência no *frontend*, implementamos indicadores visuais de carregamento durante o acionamento da IA.

4. Metodologia

O sistema foi desenvolvido em Python, utilizando o *framework* Flask para o servidor *backend* e interface web em HTML/CSS/JavaScript. O motor de raciocínio do CopaBot foi dividido em duas camadas principais:

- **Camada 1 (Regras e NLTK):** A mensagem do usuário é processada pela biblioteca NLTK. Se a intenção (ex: "buscar campeão") e as entidades (ex: "2014") forem identificadas com alta confiança, o sistema realiza uma busca exata no arquivo *copa.csv* (Base de Conhecimento Estruturada) via biblioteca *Pandas* e retorna a resposta instantaneamente.
- **Camada 2 (Motor RAG e Hugging Face):** Caso a Camada 1 não encontre parâmetros suficientes (uma pergunta ambígua ou aberta), o sistema ativa o motor RAG. Primeiramente, realiza-se uma busca vetorial no banco de dados *ChromaDB* usando *sentence_transformers* para recuperar fatos relevantes do contexto futebolístico. Em seguida, esses fatos são embutidos no *prompt* de sistema e enviados via API para o modelo generativo (optou-se pelo HuggingFaceH4/zephyr-7b-beta devido ao seu excelente balanço entre velocidade de inferência gratuita e qualidade em língua portuguesa).

5. Resultados e Discussão

Testes automatizados e manuais comprovaram a eficácia da arquitetura híbrida projetada.

- **Eficácia da Base Local (NLTK):** Perguntas diretas como "*Quem ganhou em 2002?*" foram interpretadas corretamente sem o uso de *Regex*, resultando em respostas instantâneas (<0.1s de latência) extraídas diretamente do *.csv*.
- **Ativação Correta do Fallback (RAG):** Em cenários de perguntas interpretativas, como "*Qual foi a copa mais emocionante?*", o sistema

NLTK repassou a execução com sucesso para o motor RAG. O LLM gerou respostas fluentes baseadas nos fragmentos do banco de dados vetorial, demorando em média de 3 a 5 segundos para a conclusão (período no qual o *frontend* exibiu corretamente o indicador de *loading*).

- **Redução de Alucinações:** A restrição do *prompt* no motor RAG garantiu que a IA utilizasse apenas os fatos recuperados, impedindo-a de fornecer informações futebolísticas historicamente incorretas.

6. Conclusões e Trabalhos Futuros

O projeto cumpriu integralmente os requisitos propostos, comprovando que arquiteturas híbridas (NLTK + RAG + LLM) representam o melhor equilíbrio entre eficiência computacional e inteligência conversacional. Garantimos a precisão histórica através da base estruturada e a flexibilidade dialógica através da IA Generativa.

Como **trabalhos futuros**, sugere-se a expansão da base de dados para incluir estatísticas avançadas de jogadores (*player tracking*), a implementação de reconhecimento de voz no *frontend* e a realização de um ajuste fino (*fine-tuning*) em um modelo menor (ex: Llama-3 8B) hospedado localmente, visando eliminar completamente a dependência de APIs externas de nuvem.

7. Apêndices

- **Repositório do Código-Fonte (GitHub):**
https://github.com/ItaloBM/Trabalho_Inteligencia-Artificial_Chatbot.git
- **Vídeo de Demonstração (Google Drive):**
<https://drive.google.com/file/d/1SMmwQOxHXIesVWIRGinJoh9NJk9CoqGQ/view?usp=sharing>

8. Referências Bibliográficas

1. BIRD, S.; KLEIN, E.; LOPER, E. *Natural Language Processing with Python*. O'Reilly Media, 2009. (Documentação Oficial do NLTK).
2. LEWIS, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems, v. 33, p. 9459-9474, 2020.
3. VASWANI, A. et al. *Attention is All you Need*. Advances in Neural Information Processing Systems, v. 30, 2017.
4. HUGGING FACE. *Hugging Face Hub Documentation & Inference API*. Disponível em: <https://huggingface.co/docs>. Acesso em: Junho de 2026.